

```

1 #=====
2 # MIT 6.046J 代码精读 • PS6 - 动态/有界跳 APSP
3 # 源文件: apsp.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS6 Problem 6-1: dynamic & bounded-hop All-Pairs Shortest Paths.
7
8 Nonnegative real edge weights; vertices 0..n-1; W[i][j] = weight of edge (i,j)
9 or inf if absent; W[i][i] = 0.
10
11 Implements:
12 * floyd_warshall(W)          -- standard  $O(V^3)$  APSP + predecessors;
13 * update_edge(D, Pi, W, i, j, r) -- (a) recompute D, Pi after  $w[i,j] \leftarrow r$ ;
14 * bounded_hop_fw(W, h)       -- (c) at-most-h-hop APSP by a hop-indexed FW;
15 * bounded_hop_matmul(W, h)   -- (d) via (min,+) matrix powers,  $O(V^3 \log h)$ ;
16 * bounded_hop_update(...)    -- (e) dynamic update for the bounded-hop D.
17
18 All verified against a Bellman-Ford / brute-force at-most-h-hop oracle.
19 """
20 from __future__ import annotations
21
22 import copy
23 import math
24 from typing import List, Optional, Tuple
25
26 INF = math.inf
27 Matrix = List[List[float]]
28
29
30 def floyd_warshall(W: Matrix) -> Tuple[Matrix, List[List[Optional[int]]]]:
31     n = len(W)
32     D = [row[:] for row in W]
33     Pi: List[List[Optional[int]]] = [[None] * n for _ in range(n)]
34     for i in range(n):
35         for j in range(n):
36             if i != j and D[i][j] < INF:
37                 Pi[i][j] = i
38     for k in range(n):
39         Dk = D[k]
40         for i in range(n):
41             dik = D[i][k]
42             if dik == INF:
43                 continue
44             Di = D[i]
45             for j in range(n):
46                 nd = dik + Dk[j]
47                 if nd < Di[j]:
48                     Di[j] = nd
49                 Pi[i][j] = Pi[k][j]
50     return D, Pi
51
52
53 def update_edge(W: Matrix, i: int, j: int, r: float):
54     """Part (a): change  $w[i,j]$  to  $r$  and return the new (D, Pi).
55
56     Three cases:
57     r == w[i,j]: nothing changes.
58     r < w[i,j]: a *decrease*; a single edge got cheaper. Every shortest path
59                 can only improve by routing through (i,j). We can update in
60                  $O(V^2)$ :  $D'[u][v] = \min(D[u][v], D[u][i] + r + D[j][v])$ .
61     r > w[i,j]: an *increase*; paths that used (i,j) may get longer. In the
62                 worst case this forces an  $O(V^3)$  recomputation (no better bound
63                 is possible in general -- see part (b)). We recompute via FW.
64
65     """
66     n = len(W)
67     W2 = [row[:] for row in W]
68     old = W2[i][j]
69     W2[i][j] = r
70     if r == old:
71         return floyd_warshall(W2)
72     if r < old:
73         D, Pi = floyd_warshall(W) # start from old solution
74         #  $O(V^2)$  relaxation through the improved edge (i,j)
75         for u in range(n):
76             dui = D[u][i]
77             if dui == INF:
78                 continue
79             for v in range(n):
80                 nd = dui + r + D[j][v]
81                 if nd < D[u][v]:
82                     D[u][v] = nd
83                     Pi[u][v] = Pi[j][v] if D[j][v] < INF else Pi[u][v]
84         # fix diagonal
85         for u in range(n):
86             D[u][u] = min(D[u][u], 0.0)

```

```

86         return D, Pi
87     # r > old : full recompute
88     return floyd_warshall(W2)
89
90
91 def bounded_hop_fw(W: Matrix, h: int) -> Matrix:
92     """Part (c): shortest at-most-h-hop distances via a hop-indexed DP.
93
94      $L^{\{(m)\}}[i][j]$  = shortest path from i to j using at most m edges.
95      $L^{\{(0)\}}$  = identity (0 on diagonal, inf off),  $L^{\{(1)\}}$  = W, and
96      $L^{\{(m)\}}[i][j] = \min_k L^{\{(m-1)\}}[i][k] + W[k][j]$ .
97     We iterate  $m = 1..h$ .  $O(h * V^3)$ .
98     """
99     n = len(W)
100     L = [[0.0 if i == j else INF for j in range(n)] for i in range(n)] # 0 hops
101     for _ in range(h):
102         L = _minplus(L, W)
103     return L
104
105
106 def bounded_hop_matmul(W: Matrix, h: int) -> Matrix:
107     """Part (d): same result via repeated squaring in the (min,+) semiring.
108
109      $L^{\{(h)\}} = W^{\{(h)\}}$  where multiplication is (min,+). Compute by binary
110     exponentiation of W over the (min,+) semiring:  $O(V^3 \log h)$ . Because edge
111     weights are nonnegative, allowing 'at most h' hops equals 'exactly the best
112     with  $\leq h$  hops'; padding with the identity (0-hop) matrix as the semiring
113     unit handles the 'at most' correctly.
114     """
115     n = len(W)
116     if h == 0:
117         return [[0.0 if i == j else INF for j in range(n)] for i in range(n)]
118     # result = identity (min, + unit); base = W ; but we need " $\leq h$ ", and since
119     # W has 0 on the diagonal,  $W^{\{(h)\}}$  already allows using fewer than h edges.
120     base = [row[:] for row in W]
121     result = [row[:] for row in W] #  $h \geq 1$ 
122     e = h - 1
123     while e > 0:
124         if e & 1:
125             result = _minplus(result, base)
126             base = _minplus(base, base)
127             e >>= 1
128     return result
129
130
131 def bounded_hop_update(W: Matrix, h: int, i: int, j: int, r: float) -> Matrix:
132     """Part (e): recompute the at-most-h-hop distance matrix after  $w[i,j] < r$ .
133
134     We keep enough info (just W) and recompute the h-hop matrix. Using repeated
135     squaring this is  $O(V^3 \log h)$  regardless of the direction of change -- a clean
136     bound that matches part (d). (A decrease could also be pushed in cheaper, but
137     an increase needs recomputation in the worst case, as in part (a).)
138     """
139     W2 = [row[:] for row in W]
140     W2[i][j] = r
141     return bounded_hop_matmul(W2, h)
142
143
144 def _minplus(A: Matrix, B: Matrix) -> Matrix:
145     n = len(A)
146     C = [[INF] * n for _ in range(n)]
147     for i in range(n):
148         Ai = A[i]
149         Ci = C[i]
150         for k in range(n):
151             aik = Ai[k]
152             if aik == INF:
153                 continue
154             Bk = B[k]
155             for j in range(n):
156                 v = aik + Bk[j]
157                 if v < Ci[j]:
158                     Ci[j] = v
159     return C
160
161
162 # ----- oracles -----
163 def brute_bounded_hop(W: Matrix, h: int) -> Matrix:
164     """At-most-h-hop shortest distances by Bellman-Ford-style relaxation:
165     exactly bounded_hop_fw, but written independently as the oracle."""
166     n = len(W)
167     D = [[0.0 if i == j else INF for j in range(n)] for i in range(n)]
168     for _ in range(h):
169         ND = [[D[i][j] for j in range(n)] for i in range(n)]
170         for i in range(n):
171             for k in range(n):

```

```
172         if D[i][k] == INF:
173             continue
174         for j in range(n):
175             if W[k][j] == INF:
176                 continue
177             if D[i][k] + W[k][j] < ND[i][j]:
178                 ND[i][j] = D[i][k] + W[k][j]
179         D = ND
180     return D
181
```